

Self-Propelled Arm System



Project presentation

Filip Chekalkin, Huaijing Hou, Spaska Janeva, Kamen Kanev,
Giorgi Khundadze, Xiaosheng Tan, Stoyan Vlaev, Weiye Yuan

Embedded Systems, Cyber-Physical Systems and Robotics
Technical University of Munich, Campus Heilbronn

Introduction and Outline

Overview

System summary

■ Autonomous can collection robot

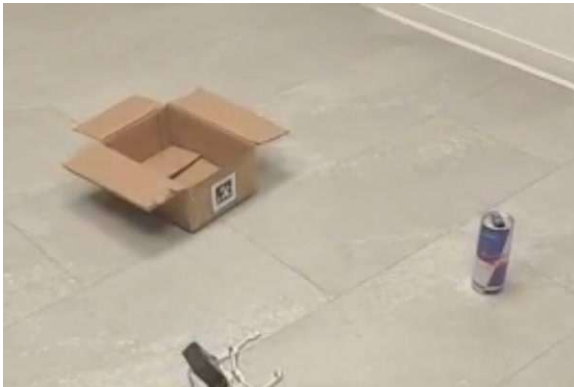
Tracked indoor platform with a four link arm and a parallel gripper

■ A single RGB camera as the only sensor

No depth sensor, no range finder, no wheel encoders, no force sensing

■ Vision in the loop at every stage

Search, approach, grasp and delivery are decided from the image



Outline



Problem and platform

Task definition and hardware constraints



Architecture and control

System components, state machine, grasping



Perception and demonstration

Detection results and a complete run on hardware



Experimental results

Detector accuracy and mission timing



Limitations and conclusion

Open issues, summary and future work

Problem Statement

Objective and constraints

- Drink cans accumulate in indoor public spaces
- Collection is repetitive, low risk manual work
- The task combines perception, navigation, decision making and manipulation



Detection

Rotate in place until the detector accepts a target



Approach

Steer on the image error, stop on apparent size



Grasp

Lower the arm, push, close, lift



Delivery

Dock on the AprilTag and release above the bin

Sensor configuration

■ Single RGB camera

■ No depth sensing

■ No wheel encoders

■ No force sensing

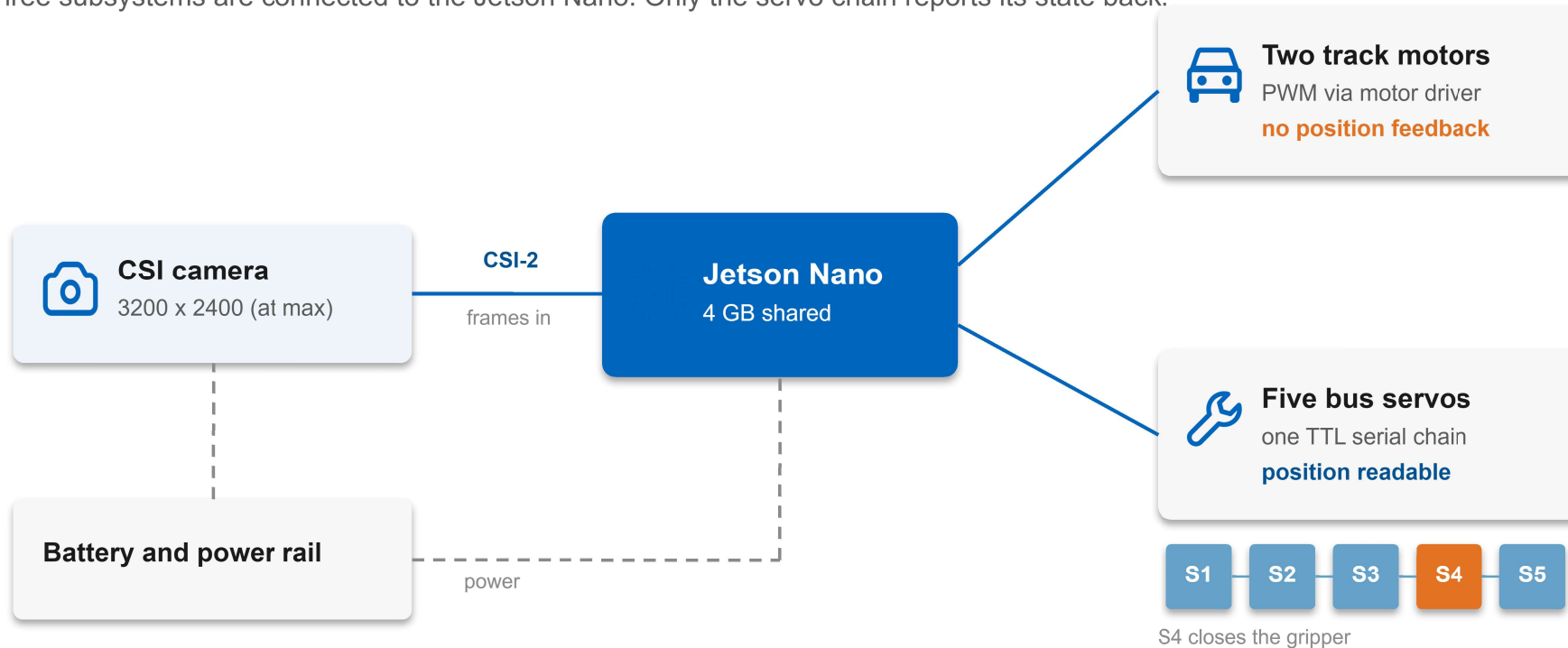
The “actors” of the show



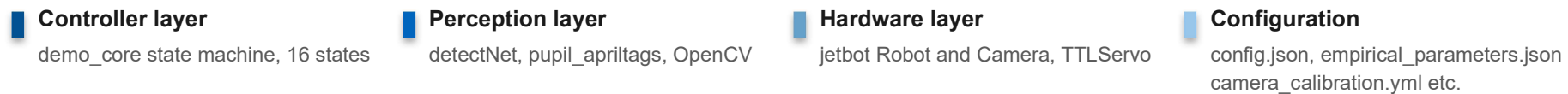
System Architecture

Components and software framework

Three subsystems are connected to the Jetson Nano. Only the servo chain reports its state back.

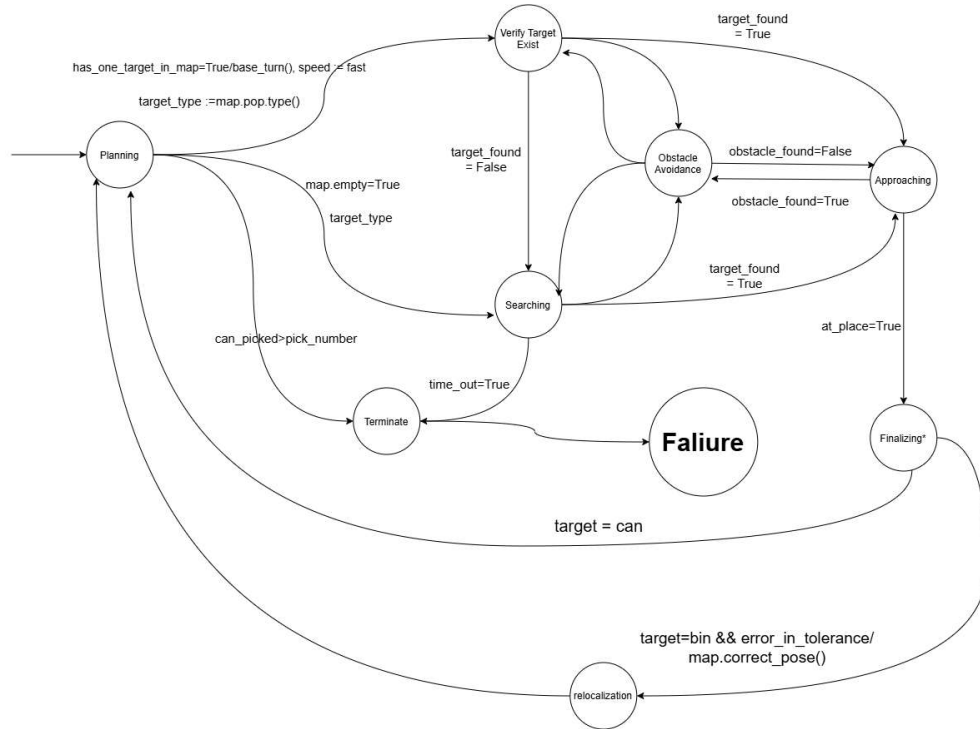


Software stack



State Machine and Control Logic

Mission control



Design level state graph. Transition guards are fields of a single shared context object.

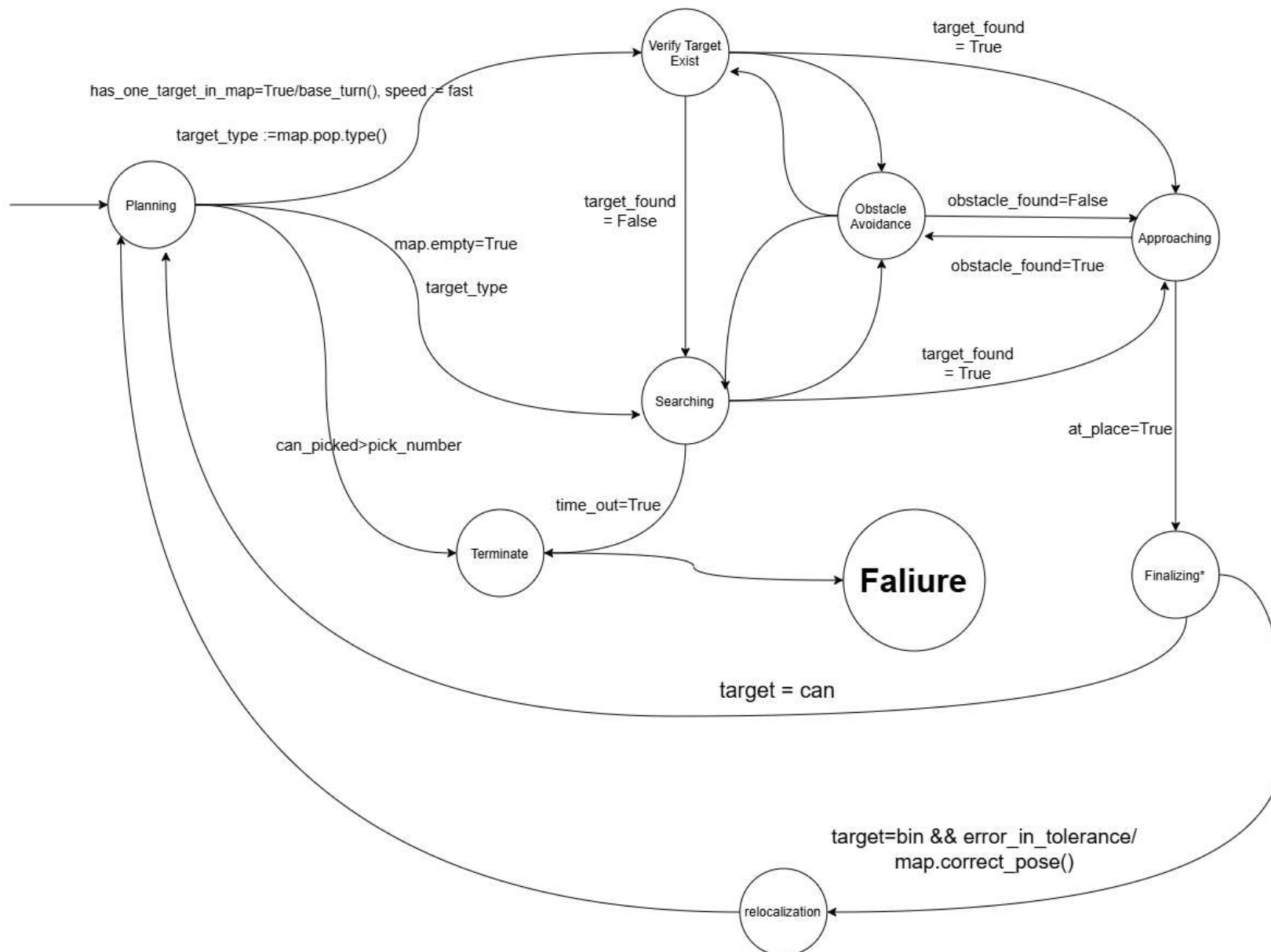
Vision driven states

- **Searching**
Rotate until a target is accepted or the timeout expires
- **Aligning**
Turn to tolerance, abort if the target is lost
- **Approaching**
Steer or drive forward, stop on apparent size
- **Final verification**
Tighter tolerance over consecutive frames

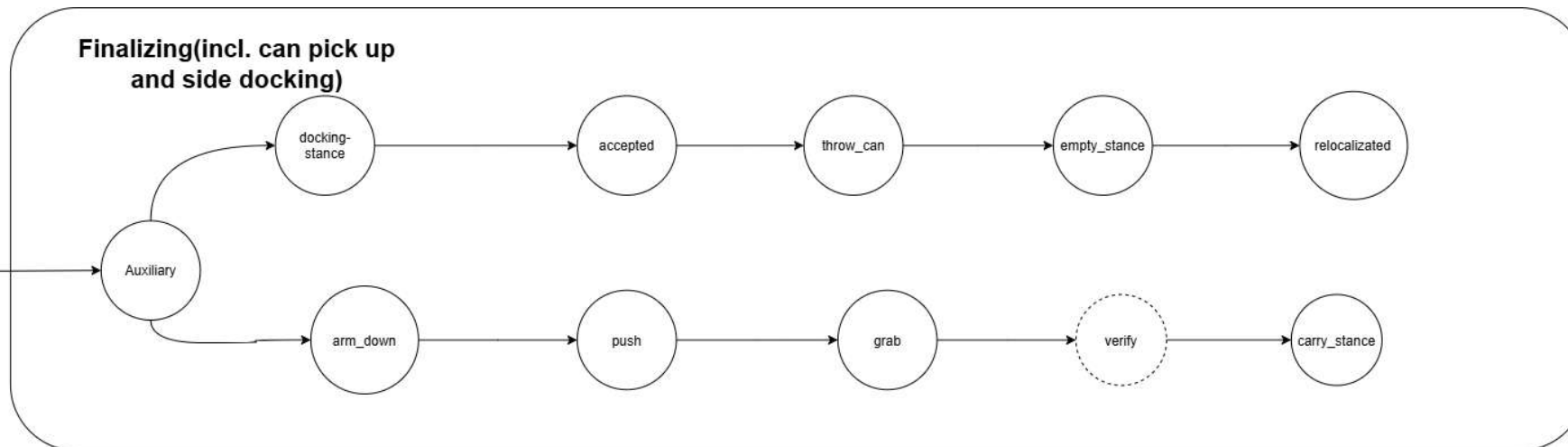
$$e_x = (x_c - W/2) / W$$

- Range -0.5 to 0.5, negative to the left of centre
- Stop thresholds: 0.36 box height, 0.15 tag height

- Transitions are held in an explicit table, so an undeclared state and event pair raises an error
- Finalising requires a stable target, a condition map based navigation never reaches, so dead reckoning triggers no action



Finalizing: abstraction of docking at bin and picking up cans

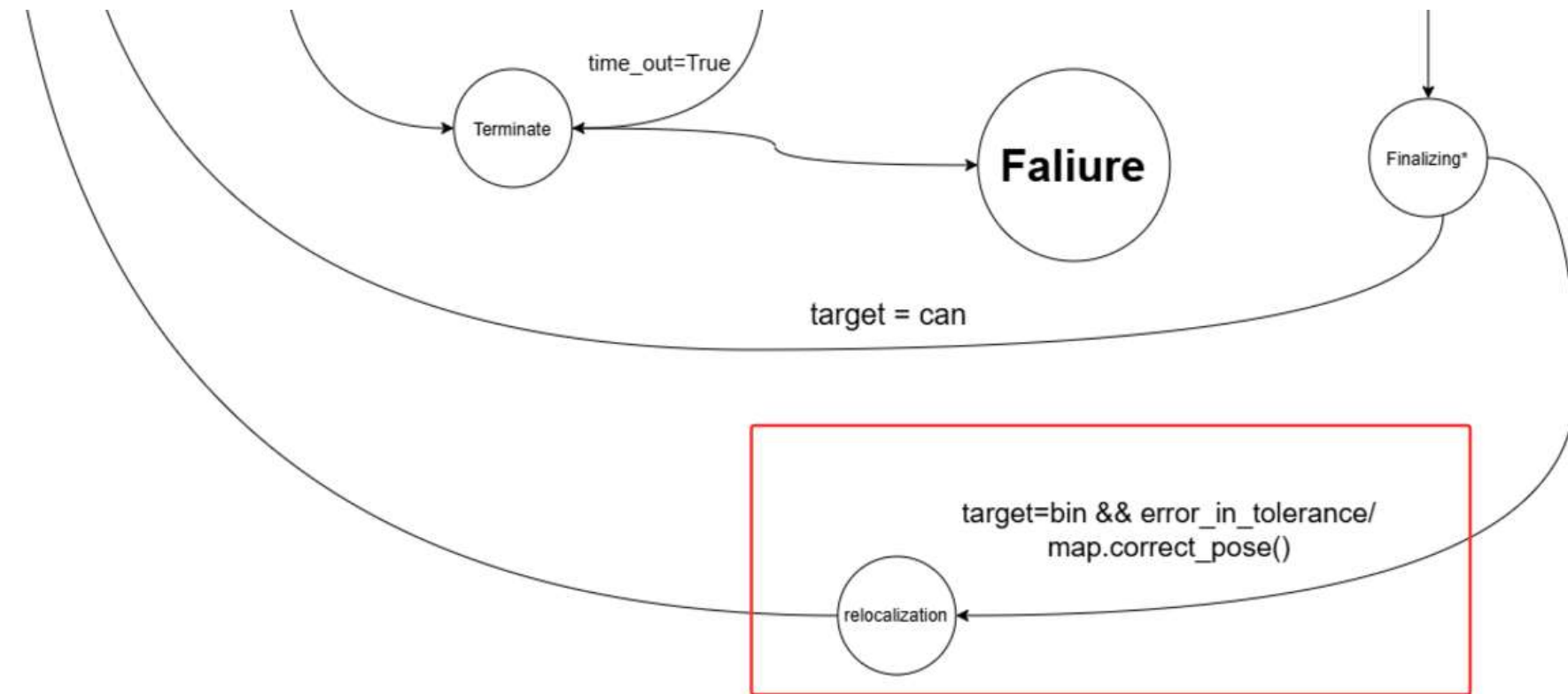


Side-docking? Why need?



The ground is a 2-dimensional space, to get back to the start point (has many benefits) as precisely as possible, we need to correct both the “z” and “x” coordinates!

Still didn't solve the whole problem..but better than no stance docking.
An ideal approach requires stronger kinematic analysis and modeling instead of just naive discrete state workaround.



The re-localization design origin(CPS slides):

Here: GPS=AprilTag, Inertial Navigation done by calibrated “speed” look up table

Difficulties in Measuring Position and Velocity

- If there is a non-zero **bias** in the measurement of acceleration, then $p(t)$ will have an error that grows proportionally to t^2
- Such an error is called **drift**, and it makes using an accelerometer alone to determine position not very useful
- However, if the position can be periodically reset to a known-good value, using for example GPS, then an accelerometer becomes useful to approximate the position between such settings

$$p(t) = p(0) + \int_0^t v(\tau) d\tau$$

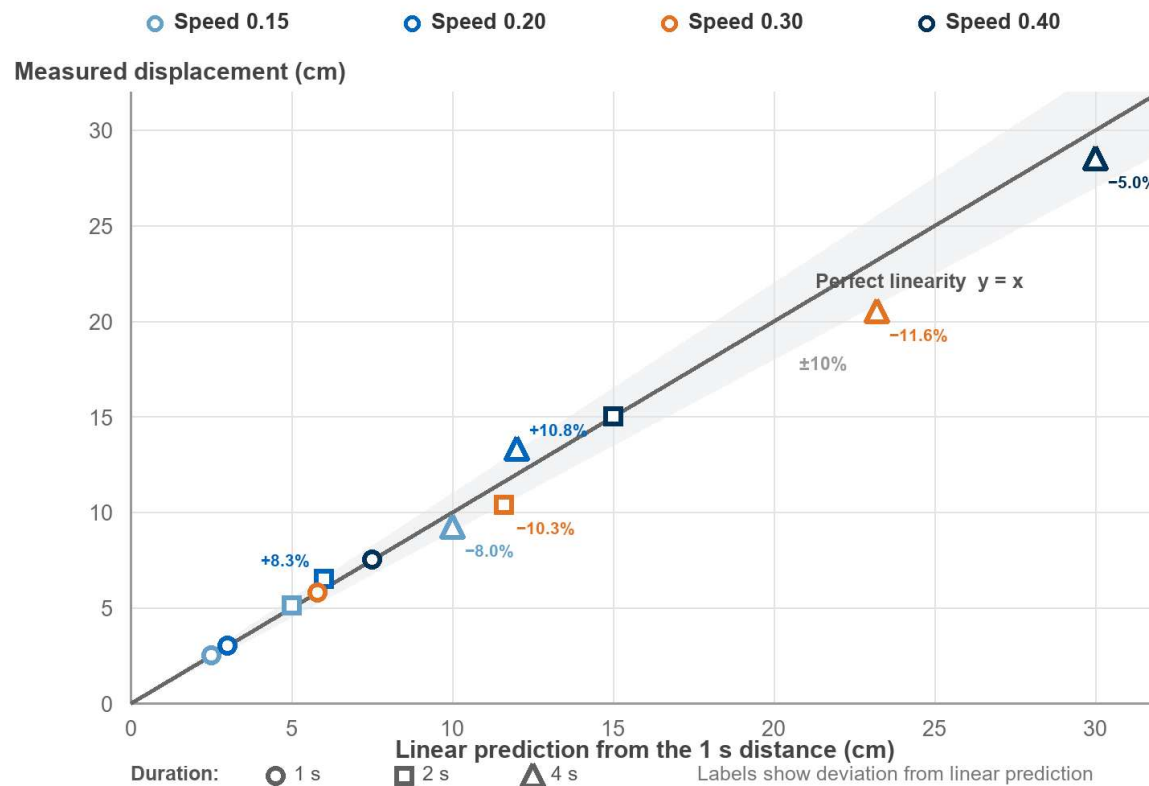
$$v(t) = v(0) + \int_0^t a(\tau) d\tau$$

robot.forward(„speed“): argument in the TTLServo function calls → implemented by Fixed Voltage.

Linear dynamic model exist?

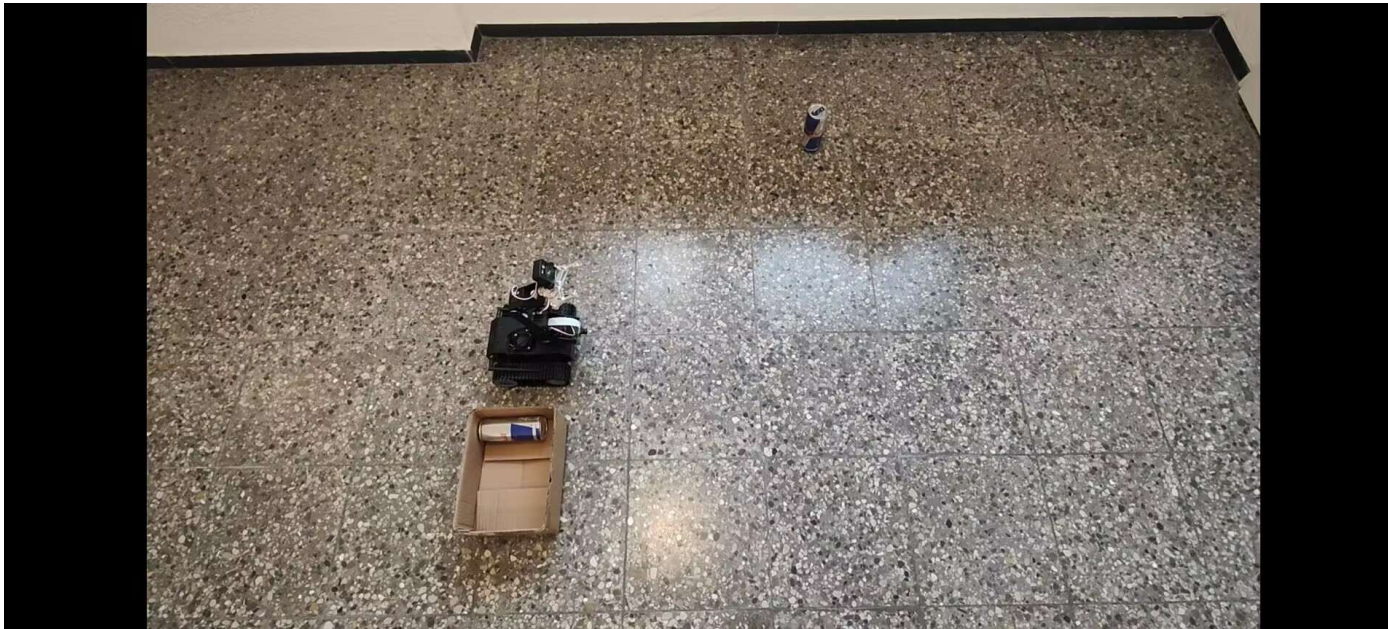
robot.forward(„speed“): argument in the TTLServo function calls → implemented by Fixed Voltage.

Linear dynamic model exist? Kind of, but for our very fragile (edge computing With modern AI) sensing system, the precision still breaks the tolerance.



Smart Solution: Taking chip temperature, friction coefficient, slipping/drift factor etc. into account

Our (Stupid) Solution: Look up table with fixed “speed” parameters and tie it to the environment e.g. the ground in the lecture hall in Weiperstrasse.

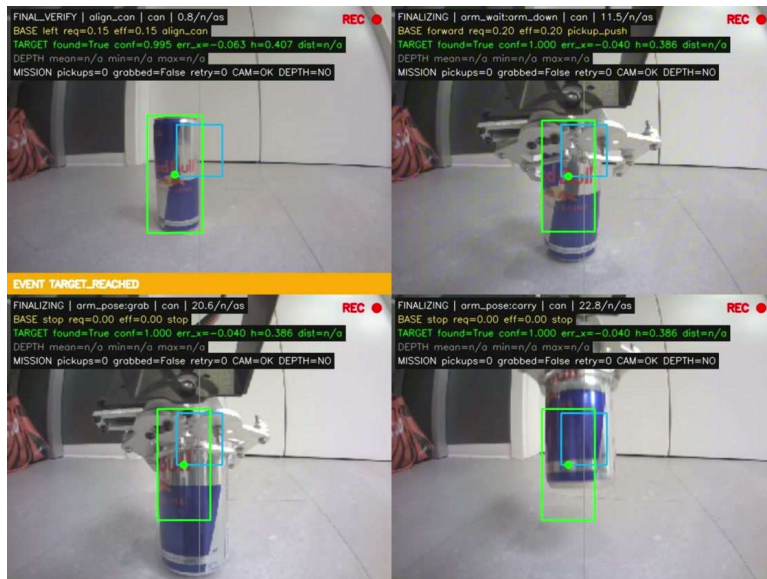
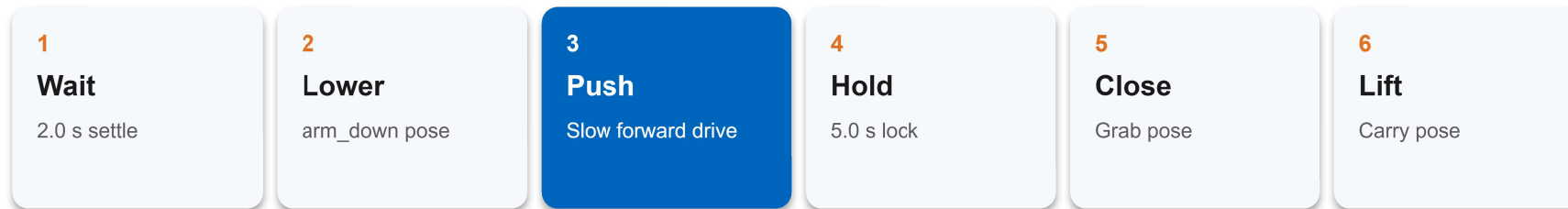


Then calibrate/test it again in somewhere else...troublesome but works.

Can Grasping Strategy

Manipulation sequence

Fixed poses — no inverse kinematics(since no sensor on the gripper)



Verify · push · close · carry

Why push, not reach

- Short arm, imprecise gripper
- Base drives reliably
- Can driven into open gripper
- Coarse motion beats fine manipulation

Servo feedback

- Polled until readings stabilise
- Push blocked until settled
- 4.94 s settle · 7.0 s push · 5.0 s hold

Experimental Setup



Platform with grasped can



Arena: can, AprilTag bin



Release above bin

73.8 s

Full run

1

Camera only

98

Can detections

41 / 59

Tag reads

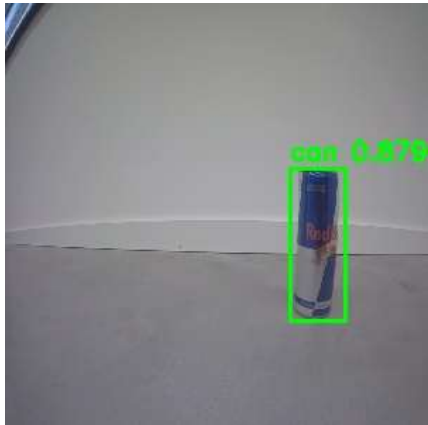
According to „some piece“ of our testing logs

Video

Autonomous Can Collection Robot

Object and Marker Detection

Two detectors, every controller input



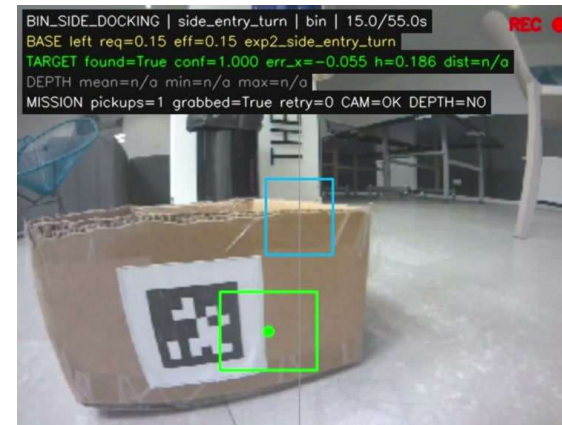
Detector output

SSD-MobileNet-V1 · 300 × 300



Can detection

error -0.119 · height 0.255



Tag detection

tag36h11 id 0 · height 0.186

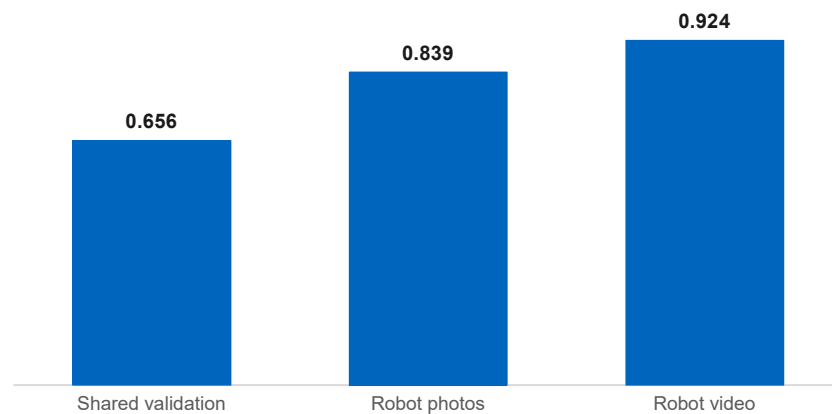
Obstacle avoidance

An experimental feature we didn't integrate. Purple is the computed tangent of the obstacle.

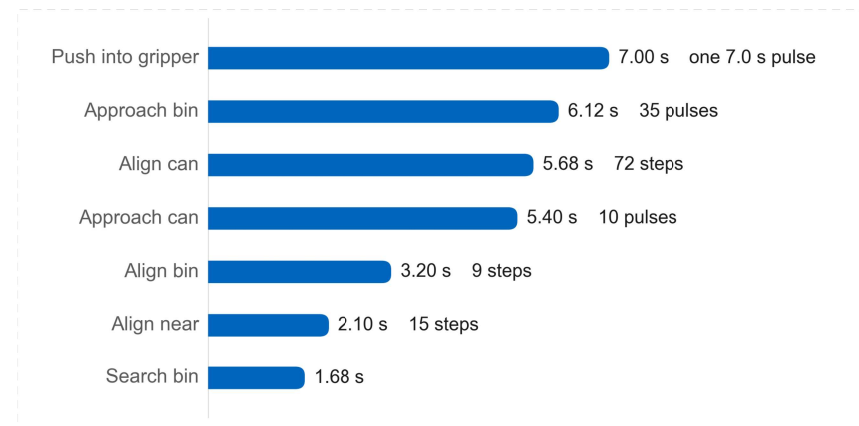


Detector Training Results

Detector F1 by evaluation set



Base motion per phase



671

Training images

194

Validation images

0.6637

Best validation mAP

Epoch 22

Selected refinement epoch

Training setup

- MobileNet V1 SSD at 300×300
- 30 initial epochs and 40 refinement epochs
- Confidence 0.20, NMS 0.30, IoU 0.50

Validation status

- ONNX checker passed at opset 11
- PyTorch vs. ONNX max error 4.4×10^{-7}
- Epoch 22 performed better across robot photo and video domains

Limitations and Challenges

Evaluation



Depth excluded

- 0.3 m general scenes
- 1 m white surfaces
- Detector reaches 3 m

Removed from control path



Pose drifts

- Left / right: 1–3 rad/s
- Modelled 0.471 / 0.942
- Error exceeds model

AprilTag: only anchor



Silent failure

- Verification disabled
- Pickup always recorded
- Empty gripper reports success

First fix: verification flag

Missing capability: relocalisation

Principal challenges

■ Open-loop turning unreliable

Pulses + visual correction

■ No gripper feedback

Full and empty identical

■ 10–15 fps

Pulsed motion, low speeds

■ Model deployment

Accuracy traded for reliability

■ Depth underperformed

Unreliable at useful ranges

■ Grasp timing

Polling beat fixed delay

Conclusion and Future Work

Summary

Summary

- Mobile manipulator, single camera
- Closed on appearance, not geometry
- Errors degrade, never break
- Drifting map lengthens search
- Asymmetric turning needs corrections

Next steps

- **Enable grasp verification**
- **Calibrate turn constants**

Sensors worth adding



Forward distance sensor

First metric measurement



Gripper force sensing

Fixes empty gripper



Wheel odometry

Map becomes measurement



More room tags

Cheap relocalisation anchors